

GO PROGRAMMING MASTER COURSE

Week 5, Day 1

HTTP, JSON, and Gin — the first SMS API you can hit from Postman.

Umesh Indrajith · BSc Eng (Hons), University of Moratuwa
Senior Software Engineer / Lecturer

SINCE LAST
WEEK

Skeleton first. Then the network.

- `go run ./cmd/server` still prints `sms` starting (or we fix that in 10 minutes).
- Module path and `internal/` tree are in place. PR from week 4 is reviewable.
- Today that process learns to speak **HTTP**.
- `gofmt` and no `.env` in git — unchanged rules.

TODAY

If Day 1 of week 5 works, you can

- Name HTTP methods, paths, and common status codes.
- Write a tiny net/http server so Gin is not magic.
- Use Gin: router, gin.Context, c.JSON.
- Return JSON with a minimal struct + tags.
- Ship GET /health on :8080 and prove it in Postman + curl.

Why REST before a full functions course?

So every remaining week has a **running product**. You copy a handler shape now; you deepen functions and structs inside SMS next.

- Spiral, not waterfall — just enough to ship a vertical slice.
- Today's milestone: health JSON. Not login. Not Postgres.
- A classmate should hit your API from Postman without reading your brain.

HTTP Client asks. Server answers.

Method

GET read · POST create · PUT replace · DELETE remove

Path

/health · /api/hello · later /api/students

Status

200 OK · 400 bad input · 404 missing · 500 server fault

Body

Often JSON. Headers carry metadata (Content - Type).

JSON The body shape APIs speak

```
{  
  "status": "ok",  
  "message": "Backend is  
}
```

- Keys are strings. Values: string, number, bool, object, array, null.
- Go turns structs ↔ JSON with `encoding/json` (and Gin helpers).
- Wrong JSON from a client → 400 later. Today we mostly *send* JSON.

LIVE
DEMOSTDLIB
FIRST

net/http Hello — 30 minutes, no Gin

```
package main
import ("fmt"; "net/http")
func main() {
    http.HandleFunc("/hello"
        fmt.Fprintln(w, "hel
    })
    http.ListenAndServe(":8080")
}
```

curl http://localhost:8080/hello
— Gin is a nicer router on top of this idea.

GIN

A router + helpers on top of net/http

- Framework rule for this course: **stdlib first, Gin for SMS.**
- You must be able to say what Gin is wrapping — routing, JSON, middleware later.
- Add the dependency once you are in backend/:

```
go get github.com/gin-gonic/gin
go mod tidy
```

GIN Router, context, JSON response

```
r := gin.Default()
r.GET("/health", func(c *gin.Context) {
    c.JSON(200, gin.H{
        "status": "ok",
        "message": "Backend is healthy"
    })
})
r.Run(":8080")
```

- gin.Context is the per-request bag: params, body, response writers.
- gin.H is map[string]any — fine for health. Prefer structs as APIs grow.

STRUCTS ·
MINIMAL

Just enough for JSON tags

```
type HealthResponse struct {
    Status string `json:"status"`
    Message string `json:"message"`
}
```

```
c.JSON(http.StatusOK, HealthResponse{
    Status: "ok",
    Message: "Backend is up and running",
})
```

Exported fields (capital letters) are what JSON can see.

Tag renames the wire name. Deep structs are week 7 — copy this pattern today.

SMS
MILESTONE

GET /health — the contract

```
GET http://localhost:
```

```
200 OK
```

```
{
```

```
  "status": "ok",
```

```
  "message": "Backend
```

```
}
```

- Matches this repo's idea in internal/routes — keep the JSON keys exact.

- Also add GET /api/hello with a small JSON message (practice).

- Listen on :8080.

LIVE
DEMO

SMS

Wire health into cmd/server

```
package main
import (
    "net/http"
    "github.com/gin-gonic/gin"
)
func main() {
    r := gin.Default()
    r.GET("/health", func(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{
            "status": "ok", "message": "Backend i
        })
    })
    r.Run(":8080")
}
```

ALSO
TODAY

GET /api/hello

```
r.GET("/api/hello", func(c) {
    c.JSON(http.StatusOK, gin.H{
        "message": "hello from api"
    })
})
```

- Groups come later; a full path is fine for day 1.
- Stretch homework: hard-coded GET /api/students JSON array — still no DB.

STATUS
CODES

Pick the number that matches the story

200

Success with a body
(health, hello).

201

Created — next weeks for
POST student.

400

Client sent junk. Your
validation said no.

404 / 500

Missing resource / you
crashed or failed
internally.

VALIDATION
• LIGHT

Reject bad input with 400

```
// idea for later POSTs – not require
if name == "" {
    c.JSON(http.StatusBadRequest, gin
    return
}
```

- Health has no body — still teach the pattern once.
- Consistent error JSON helps Postman and future frontend.
- Do not panic in a handler. Return a status and a message.

**LIVE
DEMO****CURL**

Prove it from the terminal

```
curl -i http://localhost:  
curl -s http://localhost:
```

-i shows status line and headers.

If the port is busy, someone else's server is still running — kill it or change port temporarily.

POSTMAN

The GUI you will live in

- New request → method GET → URL
`http://localhost:8080/health` → Send.
- Expect **200** and the JSON body. Screenshot for homework.
- Start a collection: SMS → folder Week 5 → requests Health, Hello.
- Save the collection; you will add Students in week 7.

CORS • MENTION
ONLY

Browsers are pickier than Postman

- Postman and curl are not browsers. They ignore CORS.
- The React app on :5173 will need CORS later (week 15).
- Do **not** implement CORS today. Know the word exists.

PROJECT
SHAPE

Where does this code live?

- **Minimum today:** routes in `cmd/server/main.go` so it runs.
- **Better:** `internal/routes` registers `/health`; `main` only builds the engine and `Run`.
- Handlers package can wait until week 6 refactor — do not block on perfect layers.

```
go run ./cmd/s  
# listen :8080
```

**LAB • REST OF
CLASS**

Done when all of this is true

- `go get Gin`; `go.mod` shows the require
- `GET /health` returns the exact JSON contract
- `GET /api/hello` returns JSON
- `curl -i` shows 200; Postman collection started
- Commit + push; show me the Postman 200 screenshot path or PR

HOMEWORK

PR + Postman proof

- PR: health + hello working on :8080.
- README: how to run + example curl.
- Attach / commit a Postman 200 screenshot (or export collection).
- **Stretch:** GET /api/students returns a hard-coded JSON array (no DB).

NEXT
SESSION

Week 6 — functions and methods inside SMS

- Extract handlers from main into a handlers package.
- defer, multiple returns — deepen, still in your API.
- Bring a green health endpoint.

WORDS TO
KEEP

Say these correctly next week

Handler

Function the router calls for a method+path.

Status code

Number telling the client how it went.

JSON

Text body shape for APIs.

Gin

Router/helpers over net/http — not magic.

SAFETY

Still true on an API week

- No secrets in git. `.env` stays ignored.
- Do not commit the binary from `go build`.
- Port 8080 conflict: find and stop the old process.
- You are not graded on React today — Postman is enough.

Questions